

EPF AFRICA

DOSSIER ARCHITECTURAL ET TECHNIQUE

NOTIFICATIONS, SECURITE, DASHBOARD ADMIN ET MECANISMES FRONTEND

Ce document constitue la référence absolue pour le développement, la maintenance et l'évolution de la plateforme EPF Africa. Il détaille l'ensemble des modules, les structures de requêtes, le dictionnaire d'API sécurisé, les relations SQL, l'explication des packages utilisés et la configuration réseau Axios.

Metadonnées du Projet :

Destinataire : Développeurs du Projet (Pair Programming)
Framework Backend : Laravel 10 (Rest API + Sanctum + Eloquent)
Framework Frontend : React 19 (Vite, Framer Motion, Tailwind CSS)
Modélisation BDD : PostgreSQL / SQLite (Contraintes d'intégrité)
Version du document : 3.0 (Spécifications complètes)

1. Préambule Pédagogique du Développeur Senior

Salut champion, et bienvenue sur le projet EPF Africa !

Si tu lis ce guide, c'est que tu t'apprêtes à te plonger dans le code de notre plateforme de gestion académique. Ne panique pas, la structure est très logique. L'application est découpée en deux mondes indépendants : un backend Laravel faisant office d'API REST robuste et un frontend React hyper fluide pour une expérience utilisateur de niveau international.

Dans ce document, je vais t'expliquer en détail le rôle de chaque couche de l'application, le fonctionnement des packages installés, la structure des requêtes et notre protocole de communication asynchrone. C'est l'essence même de notre produit. Prends un café, lis attentivement et garde ce document ouvert à côté de ton terminal !

2. Principes Fondamentaux d'une Requête API

Toutes nos requêtes HTTP sont asynchrones et s'appuient sur le format JSON. Pour sécuriser l'accès, nous utilisons des jetons de session émis par Laravel Sanctum.

Lorsqu'une requête part du frontend, elle doit obligatoirement inclure l'en-tête suivant si elle est sécurisée :
Authorization: Bearer <jeton_sanctum>

Voici le schéma d'une requête classique de notification :

```
METHOD: POST
URL: http://localhost:8000/api/notifications/tout-marquer-lu
HEADERS:
- Accept: application/json
- Content-Type: application/json
- Authorization: Bearer 8|k8zP98xQLmPqY2...
RESPONSE STATUS: 200 OK
RESPONSE BODY:
{
  "message": "Toutes les notifications ont été marquées comme lues."
}
```

3. Description des Couches Applicatives du Backend (Laravel)

Notre architecture backend Laravel repose sur une décomposition stricte en couches logiques. Voici le rôle de chacune de ces couches :

A. Couche de Routage (routes/api.php)

Elle fait office d'aiguilleur d'URL. C'est ici que l'on déclare les points d'accès (endpoints) de l'API. Nous utilisons des groupes de routes protégés par le middleware 'auth:sanctum' pour garantir l'authentification.

B. Couche de Validation (app/Http/Requests)

Sécurité avant tout. Laravel utilise des objets FormRequest pour intercepter les requêtes entrantes avant qu'elles n'atteignent le contrôleur. Cela garantit que les types de données, les plages et les contraintes (ex: 'required', 'email', 'exists') sont respectés.

C. Couche de Contrôle (app/Http/Controllers/Api)

Les contrôleurs valident la transaction. Ils récupèrent les requêtes propres, délèguent les calculs ou actions à la couche Service, puis transforment le résultat en réponse JSON. Un bon contrôleur ne dépasse jamais 50 lignes.

D. Couche de Logique Métier (app/Services)

La couche Services isole les traitements algorithmiques complexes (ex: calculs de moyennes de notes, archivages automatiques, envois de mails, notifications). Cela évite les doublons de code dans l'application.

E. Couche d'Autorisation (app/Policies)

C'est la sécurité des ressources. Les Policies comparent les rôles et identifiants de l'utilisateur connecté avec l'entité ciblée. Par exemple, un étudiant ne peut pas modifier un stage qui appartient à un autre étudiant.

F. Couche Modèle & Persistance (app/Models)

Mappe nos tables SQL. Grâce à l'ORM Eloquent, chaque modèle PHP représente une entité métier (Soutenance, Memoire, User, etc.) et gère automatiquement ses clés étrangères et relations d'intégrité.

4. Description des Couches Applicatives du Frontend (React)

Notre architecture React est conçue de manière modulaire afin de faciliter l'autonomie des développeurs sur chaque module utilisateur :

A. Couche de Routage Client (App.jsx & ProtectedRoute.jsx)

Gère les URL virtuelles du navigateur. L'utilisation de react-router-dom permet de charger instantanément les pages métiers. La couche ProtectedRoute vérifie le state d'authentification globale avant d'accorder l'accès.

B. Couche Modules Métiers (src/modules/*)

Le code est partitionné par typologie d'utilisateurs : 'etudiant', 'encadreur', 'jury', 'administration'. Chaque module contient ses propres sous-dossiers '/pages' et '/components' afin de maximiser l'isolation du code.

C. Couche de Composants Partagés (src/shared/components)

Héberge les éléments communs de la plateforme (le Layout avec Sidebar adaptative, la cloche NotificationBell, les spinners, et formulaires de base). Ils respectent tous notre identité Rose & White.

D. Couche de Service Réseau (src/shared/api)

Déclare nos clients d'API (Axios). Les fonctions de requêtes réseau y sont isolées et structurées (ex: `stagesApi.getAll()`) afin d'éviter d'écrire des requêtes 'fetch' brutes au milieu de nos composants React.

E. Couche CSS & Tokens de Style (theme.css)

Fichier CSS natif déclarant l'ensemble des variables de couleur, de rayon de bordure et de transitions du design (variables CSS `--surface-raised`, `--border`, `--ink`, etc.).

6. Répertoire Général des APIs - Partie 1

Toutes les communications frontend-backend s'effectuent par ces endpoints d'API :

POST /api/login

Authentification. Payload : {email, password}. Retourne le modèle User complet avec ses rôles/permissions, et le token d'accès Sanctum sous forme de chaîne de caractères.

POST /api/logout

Déconnexion sécurisée. Middleware : auth:sanctum. Révoque et supprime le jeton d'accès courant dans la table `personal\_access\_tokens`.

GET /api/me

Récupération du profil. Retourne l'utilisateur connecté avec ses rôles et permissions.

GET /api/stages

Liste des stages. Accessible à l'administration et aux encadreurs. Retourne un tableau paginé des stages en cours ou terminés avec relations entreprise et étudiant chargées.

POST /api/stages

Création de stage. Réservé à l'administration. Payload : {etudiant_id, entreprise_id, titre, description, date_debut, date_fin}.

POST /api/stages/{stage}/valider

Validation de la convention de stage par l'administration. Déclenche une notification in-app pour l'étudiant.

GET /api/encadrements

Liste des encadrements actifs de l'enseignant connecté. Retourne le lien étudiant, les comptes-rendus et les rendez-vous.

POST /api/encadrements/{encadrement}/rendez-vous

Planification d'un rendez-vous par l'encadreur. Payload : {date_rendez_vous, heure, description, lieu}.

7. Répertoire Général des APIs - Partie 2

GET /api/memoires

Liste des sujets de mémoires. Supporte les filtres par statut ('propose', 'en_cours', 'valide_final').

POST /api/memoires

Proposition ou attribution de sujet. Si l'enseignant soumet, le mémoire passe directement à 'en_cours'. Si l'étudiant soumet, il passe au statut 'propose' en attente de validation.

POST /api/memoires/{memoire}/valider

Validation du sujet proposé par l'étudiant. Action réservée à l'encadreur assigné.

POST /api/memoires/{memoire}/versions

Dépôt d'une nouvelle version de mémoire par l'étudiant. Payload : multipart/form-data contenant le fichier PDF. Crée une ligne dans `memoire\_versions`.

POST /api/versions/{version}/corriger

Annotation et notation d'une version de mémoire par l'encadreur. Met à jour le pourcentage d'avancement et insère les remarques.

POST /api/versions/{version}/valider-finale

Validation finale du mémoire par l'encadreur. Le statut du mémoire passe à 'valide_final' rendant l'étudiant éligible pour réserver une soutenance.

POST /api/soutenances

Planification d'une soutenance par l'administration. Payload : {memoire_id, date_soutenance, heure_debut, salle}.

POST /api/soutenances/{soutenance}/jury

Affectation de la composition du jury (président, rapporteur, examinateur) par l'administration. Payload : {membres: [{user_id, role_jury, date_debut_acces, date_fin_acces}]}.

POST /api/soutenances/{soutenance}/publier-resultats

Délibération et publication finale par l'administration. Calcule la moyenne globale, verrouille la soutenance en 'terminee' et envoie la notification de relevé à l'étudiant.

8. Répertoire Général des APIs - Partie 3

GET /api/mon-espace/stages

Espace Étudiant. Récupère l'historique complet des stages de l'étudiant connecté.

GET /api/mon-espace/resultats-soutenance

Espace Étudiant. Récupère les notes de soutenance si le flag `resultats\_publies` est vrai. Sinon, renvoie une erreur 403 Forbidden.

GET /api/jury/mes-soutenances

Espace Jury. Récupère la liste des soutenances auxquelles le juré authentifié est affecté pour évaluation.

POST /api/soutenances/{soutenance}/notes

Saisie des notes par un juré. Payload : {notes: [{critere, note, commentaire}]}. Crée ou met à jour les critères d'évaluation de ce juré.

POST /api/jury/soutenances/{soutenance}/valider-notes

Verrouillage définitif des notes du juré. Change le flag pivot `notes\_validees` en true. Non réversible.

GET /api/notifications

Récupère le flux de notifications de l'utilisateur authentifié (paginé, trié par date de création décroissante).

POST /api/notifications/{notification}/lu

Marque une notification spécifique comme lue en renseignant la colonne `date\_lecture`.

GET /api/dashboard/apercu

Retourne les statistiques clés pour le tableau de bord (nombre de mémoires par statut, taux de validation, alertes de retards).

9. Le Système et l'Affichage des Notifications

Le système de notifications in-app est conçu pour offrir une interactivité en temps réel entre les étudiants, les encadreurs et l'administration.

A. Modélisation SQL & Relations Polymorphiques :

La table `notifications` possède une structure polymorphique. Au lieu de lier physiquement une notification à une seule table (comme stages ou mémoires), elle utilise deux colonnes :

- `notifiable\_type` : Le namespace du modèle Laravel (ex: 'App\Models\Stage').
- `notifiable\_id` : L'ID de l'enregistrement lié (ex: ID du stage).

Grâce à cela, nous pouvons rattacher n'importe quelle notification à n'importe quel objet métier de l'application.

B. La Cinématique Côté Backend :

Le `NotificationService` contient toutes les méthodes pour émettre les alertes. Par exemple, lors de la publication d'une note, la méthode `resultatsPublies(Soutenance \$soutenance)` est appelée. Elle crée une ligne dans la table en liant le `user\_id` de l'étudiant concerné.

C. L'Affichage Côté Frontend (React) :

1. Au montage de la vue générale (Layout.jsx), le composant `NotificationBell` effectue un appel API initial vers `GET /api/notifications/non-lues/count` pour afficher une bulle rouge sur la cloche.
2. Un intervalle cyclique (polling) ré-interroge cette route toutes les 30 secondes pour mettre à jour la bulle.
3. Au clic sur la cloche, l'état `isOpen` bascule à true, ce qui déclenche le chargement des 10 dernières notifications non lues via `GET /api/notifications?statut=non\_lue`.
4. Au clic sur une notification de la liste, le frontend envoie une requête `POST /api/notifications/{id}/lu` pour la marquer comme lue, puis redirige instantanément l'utilisateur vers la page concernée grâce aux données de contexte.

10. La Sécurité Globale de l'Application

La sécurité d'EPF Africa est bâtie sur quatre piliers défensifs répartis entre le backend et le frontend :

A. Authentification Stateless (Laravel Sanctum) :

Le serveur ne stocke pas de cookies ou de sessions PHP traditionnelles. Lors du login, le jeton généré par Sanctum est stocké dans le `localStorage` du navigateur. Ce jeton est transmis dans chaque header HTTP. Les jetons inactifs ou révoqués sont rejetés par le middleware `auth:sanctum` avec un statut 401.

B. Contrôle d'Accès basé sur les Rôles (RBAC) :

Chaque utilisateur se voit attribuer un ou plusieurs rôles (`etudiant`, `enseignant_encadreur`, `jury_soutenance`, `administration`). Les routes de l'API Laravel sont protégées par le middleware de rôles. Par exemple, un étudiant tentant d'appeler une route `/api/administration/*` reçoit immédiatement un code 403 Forbidden.

C. Autorisations Fines au niveau des Ressources (Laravel Policies) :

Les rôles ne suffisent pas à empêcher les abus (ex: un encadreur pourrait tenter de noter un mémoire qu'il n'encadre pas). Laravel utilise des Policies pour valider l'ownership (la propriété) de la donnée. La Policy vérifie en base de données que le `encadreur_id` du mémoire correspond bien à l'ID de l'utilisateur connecté.

D. Validation stricte et nettoyage des données (Form Requests) :

Toutes les saisies utilisateur sont inspectées avant d'exécuter une requête SQL ou d'écrire un fichier. Les injections SQL sont bloquées par défaut par l'utilisation de requêtes préparées (Eloquent PDO). Les scripts XSS sont désactivés par le nettoyage des chaînes de caractères au niveau des requêtes de formulaires.

11. Implémentation du Tableau de Bord de l'Administration

Le tableau de bord d'administration est le centre névralgique pour le pilotage de la scolarité. Il a été conçu pour charger des indicateurs en temps réel sans ralentir la base de données.

A. Architecture Backend (DashboardController.php) :

Le contrôleur expose plusieurs routes spécialisées pour alimenter les différents panneaux du dashboard :

- `/api/dashboard/aperçu` : Récupère les métriques globales (totaux, mémoires en cours, validés, soutenus, soutenances planifiées, etc.). Il s'appuie sur des requêtes SQL de comptage rapide (count).
- `/api/dashboard/graphiques` : Formate les statistiques sous forme de listes d'objets standardisés `[{label, value}]` (ex: répartition des mémoires par statut, moyenne générale). Ce format est directement exploitable par les bibliothèques de charts en frontend.
- `/api/dashboard/delais-moyens` : Calcule le temps moyen en jours écoulé entre le dépôt initial du sujet et la soutenance finale.

B. Système d'Exports Multiformats :

L'administration peut exporter les données à des fins d'archivage ou de délibération :

- **Export PDF** (Laravel dompdf) : Compile une vue HTML en document A4 élégant.
- **Export Excel** (maatwebsite/excel) : Génère des fichiers tableurs complexes.
- **Export CSV** (PHP native stream output) : C'est une implémentation haute performance. Au lieu d'écrire un fichier sur le disque du serveur, le backend écrit les lignes directement dans le flux de sortie HTTP (`php://output`) avec un encodage UTF-8 BOM pour Excel. Le téléchargement commence instantanément sans consommer de RAM.

C. Rendu Frontend (React) :

Le composant `AdministrationDashboard.jsx` récupère les données via Axios au montage de la page. Les cartes de statistiques KPI et les graphiques Recharts s'affichent de façon fluide avec des micro-animations Framer Motion.

12. Le rôle du Hook `useNavigate()` dans une Single Page Application

Le hook `useNavigate()` est une fonctionnalité majeure fournie par la bibliothèque `react-router-dom` (versions 6+).

A. Pourquoi ne pas utiliser des liens HTML standards (`<a href= "... ">`) ?

Dans une application Web classique, cliquer sur un lien `<a href="/dashboard">` demande au navigateur de charger une nouvelle page. Le navigateur détruit alors toute la mémoire de l'application (le state React, les variables stockées en cache, etc.), télécharge à nouveau les fichiers HTML, CSS, JavaScript et rétablit la connexion.

Dans une Single Page Application (SPA), nous voulons naviguer de façon instantanée sans détruire l'état de l'application. C'est là qu'intervient le routage client.

B. Le Rôle de `useNavigate()` :

`useNavigate()` retourne une fonction JavaScript qui permet de modifier l'adresse URL du navigateur et de changer le composant affiché à l'écran **sans aucun rechargement de page**.

C. Cas d'usages courants dans le projet :

1. Redirection programmatique après une action : Lors de la soumission du formulaire de connexion, si l'authentification réussit, nous appelons `navigate('/espace-etudiant')` pour diriger l'utilisateur vers son tableau de bord.
2. Retour arrière : L'appel `navigate(-1)` permet de simuler un clic sur le bouton 'Retour' du navigateur de l'utilisateur, très utile sur nos fiches de détails de dossiers.
3. Redirection de sécurité : Si l'intercepteur Axios intercepte une erreur 401 (token expiré), il nettoie les jetons et appelle `navigate('/login')` pour forcer l'utilisateur à se reconnecter.

13. Les packages installés et leur utilité

Pour éviter de réinventer la roue et accélérer les développements, nous utilisons plusieurs packages officiels et communautaires de confiance :

A. Du côté Backend Laravel (composer.json) :

- laravel/sanctum

Gère l'émission et la validation des jetons d'accès API. C'est lui qui permet de délivrer un token unique lors de la connexion et de vérifier sa validité à chaque requête sécurisée.

- spatie/laravel-permission

Permet de gérer finement l'attribution des rôles (etudiant, jury_soutenance, administration, enseignant_encadreur) et leurs permissions associées en base de données.

- barryvdh/laravel-dompdf

Une surcouche pour charger des templates HTML et les convertir automatiquement en fichiers PDF téléchargeables (par exemple pour générer les procès-verbaux de soutenance).

- maatwebsite/excel

Gère les imports et exports de fichiers au format Excel ou CSV. Très utile pour que l'administration puisse exporter la liste des notes finales ou des étudiants.

B. Du côté Frontend React (package.json) :

- react-router-dom

Gère le système de navigation client. Il permet d'associer des composants aux URL (/espace-etudiant, /espace-jury, etc.) et de naviguer sans rechargement de page.

- framer-motion

Bibliothèque d'animations premium. Elle orchestre les animations au montage des pages et le déplacement des éléments (ex: le slide actif de la sidebar).

- lucide-react

Une magnifique collection d'icônes vectorielles (SVG) épurées et légères, intégrables sous forme de composants React.

- tailwindcss & @tailwindcss/vite

Framework CSS utilitaire. Il permet de mettre en page notre application à l'aide de classes utilitaires directement injectées au build de l'application.

14. Le fonctionnement d'Axios en profondeur

Axios est le client HTTP le plus populaire pour le développement web. Contrairement à la fonction native Fetch de Javascript, Axios offre des fonctionnalités prêtes à l'emploi essentielles à notre projet :

1. Gestion automatique du JSON : Axios sérialise automatiquement les payloads envoyés et parse les réponses reçues en objets exploitables par React.
2. Les Intercepteurs (Interceptors) : C'est notre arme secrète. Nous avons configuré un intercepteur de requête qui récupère le token Sanctum dans le stockage du navigateur et l'injecte dans le header HTTP de chaque appel.
3. L'interception d'erreur : Si le serveur renvoie une erreur d'authentification (code 401), un intercepteur de réponse déconnecte automatiquement l'utilisateur et le redirige vers la page de login.

Voici à quoi ressemble notre code d'intercepteur Axios :

```
import axios from 'axios';

const apiClient = axios.create({
  baseURL: 'http://127.0.0.1:8000/api',
  timeout: 10000,
});

// Intercepteur de requête : injecte le token Sanctum
apiClient.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Intercepteur de réponse : gère les déconnexions forcées
apiClient.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response && error.response.status === 401) {
      localStorage.removeItem('token');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);
```

15. Conclusion & Conseils de développement

En respectant cette séparation stricte en couches et en comprenant le rôle de chaque package, tu éviteras d'introduire des bugs de sécurité (grâce aux Policies) ou des dysfonctionnements d'affichage (grâce à nos intercepteurs Axios et la gestion fine du state dans React).

Voici trois règles d'or à garder à l'esprit pendant que tu codes :

1. Ne mets jamais de logique métier complexe dans les contrôleurs. Utilise les Services.
2. Valide systématiquement toutes les données entrantes via des formulaires de requête sécurisés.
3. Pense à gérer les cas d'erreur de chargement (loaders, catch d'erreurs) côté frontend pour éviter les écrans figés pour l'utilisateur.

Voilà champion, tu as maintenant toutes les clés en main pour comprendre et coder sur EPF Africa. Bon code !